Spectral Bridges

Sequential C++ Implementation

Phase 1 of CPU-GPU Parallelization Project

Sirjan Hansda | Yuvaraj DC

1 High-Level Overview

Spectral Bridges is a clustering algorithm that combines k-means and spectral clustering to find non-convex, complex cluster structures in large datasets efficiently. This document describes the sequential C++ implementation — the baseline that will be parallelised using CUDA in the next phase of the project.

1.1 What the algorithm does

Given a dataset of n points, Spectral Bridges finds k clusters in four steps:

1. Vector quantization — divide the data into m small Voronoi regions using k-means++.
2. Affinity computation — for every pair of regions, measure how much data density exists between them (the 'bridge' concept).
3. Spectral clustering — build a graph from those affinities, compute its Laplacian eigenvectors, and run k-means on them to find k clusters of regions.
4. Label propagation — assign each original data point the label of its Voronoi region.

The key insight is that instead of running spectral clustering on all n points (which is $O(n^3)$), it runs on only m regions ($O(m^3)$), where $m << n$. This makes it scalable to large datasets.

1.2 Why a sequential C++ baseline first

The Python reference implementation already uses optimised C backends (FAISS, BLAS, LAPACK) so comparing Python speed to CUDA speed would be misleading — it would conflate language overhead with parallelisation benefit. The correct comparison is:

Version	Purpose
Python original	Correctness oracle only — verify outputs match
C++ sequential (this document)	Timing baseline — the 'before' number
C++ CUDA (Phase 2)	Timing result — the 'after' number

The speedup reported in Phase 2 will be C++ sequential time divided by CUDA time, which honestly isolates what GPU parallelisation contributes.

1.3 Project structure

```
spectral_bridges_cpp_impl/
  include/
    kmeans.hpp          -- KMeans class declaration
    affinity.hpp        -- computeAffinity() declaration
    spectral.hpp        -- SpectralBridges pipeline declaration
  src/
    kmeans.cpp          -- k-means++ init + Lloyd iterations
    affinity.cpp        -- bridge affinity matrix computation
    spectral.cpp        -- Laplacian, eigen, full pipeline
  tests/
    generate_reference.py   -- Python script: saves reference data
    test_kmeans.cpp         -- unit test for k-means
    test_reference.cpp      -- ARI/NMI correctness vs Python
    data/                   -- generated CSV datasets + metrics
  main.cpp                  -- quick smoke test
  CMakeLists.txt            -- build configuration
```

1.4 How to build and run

Prerequisites: macOS with Xcode command-line tools, CMake (brew), Eigen3 (brew). No additional installs needed — Accelerate framework is built into macOS.

```
# Clone / enter project directory
cd spectral_bridges_cpp_impl

# Build
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j4

# Run smoke test
./spectral_bridges
```

```
# Run unit tests
./test_kmeans

# Generate Python reference data (run once from project root)
cd ..
python tests/generate_reference.py

# Run correctness tests
./build/test_reference
```

1.5 Results summary

All four datasets pass correctness checks. The C++ output closely matches both the ground truth labels and the Python reference output:

Dataset	n	k	m	C++ vs Truth ARI	C++ vs Python ARI	Status
blobs	300	3	30	1.000	1.000	PASSED
moons	200	2	20	1.000	1.000	PASSED
circles	200	2	20	1.000	1.000	PASSED
large	1000	5	50	0.973	0.993	PASSED

The small gap on the large dataset is expected — k-means++ uses random seeding so two independent runs (Python vs C++) will differ slightly. Both achieve the same quality level relative to ground truth.

2 Deep Dive — Algorithm and Implementation

This section is for readers who want to understand the code in detail. It covers each component, the key design decisions, and the translation choices made when porting from Python.

2.1 K-Means++ (kmeans.hpp / kmeans.cpp)

What it does

K-means partitions n points into m clusters by alternating between two steps: assigning each point to its nearest centroid, and recomputing centroids as cluster means. The ++ variant uses a smarter initialisation (k-means++) that picks initial centroids spread apart, giving much better and more reproducible results than random initialisation.

Key data structure

All matrices are stored as flat std::vector<float> in row-major order. Element at row i, column j of a matrix with 'cols' columns is accessed as data[i * cols + j]. This matches numpy's default layout and makes it straightforward to later pass to CUDA kernels which expect the same layout.

Initialisation (initCentroids)

Translates numpy's rng.choice(..., p=dists/inertia) into C++ using std::discrete_distribution<int>. The logic is identical to the Python version:

- Pick the first centroid uniformly at random.
- Maintain a minDists array — the squared distance from each point to its nearest chosen centroid so far.
- Sample 'trials' candidate centroids proportional to minDists (points far from existing centroids are more likely to be picked).
- Pick the candidate that minimises total inertia if added.
- Repeat until m centroids are chosen.

Lloyd iterations (fit)

Standard assignment + update loop. One subtlety: if a cluster becomes empty during an iteration, the old centroid is kept rather than crashing. This matches FAISS behaviour and prevents degenerate cases on small datasets.

Python equivalence note

The Python version delegates the actual k-means training to FAISS (a C++ library), using k-means++ only for initialisation. Our C++ version reimplements the full Lloyd loop directly. Both produce equivalent results because Lloyd iterations are deterministic given the same initial centroids.

2.2 Bridge Affinity Matrix (affinity.hpp / affinity.cpp)

The geometric idea

For two Voronoi regions Vk and Vl with centroids μk and μl, consider the line segment (bridge) connecting them. For each point xi in Vk, compute how far it projects along this bridge:

```
ti = dot(xi - μk,  μl - μk) / ||μl - μk||^2   (clamped to [0, 1])
```

If ti is large, the point is close to the midpoint of the bridge — it sits in the gap between the two regions. If ti is near zero, the point hugs its own centroid and there is a low-density gap. The affinity between two regions is the sum of ti^p values (default p=2) over all points in the region, normalised by region size. Large affinity means the two regions should be merged into the same cluster.

Implementation walkthrough

Step 1 — Center each region: subtract the centroid from every point in that region. This is the X_centered computation. Storing these per-region avoids recomputing the subtraction in the inner loop.

Step 2 — Outer loop over regions i: for each region i, compute the segment vectors from μi to every other centroid μj, and their squared lengths (dists). Set dists[i] = 1 to avoid division by zero on the diagonal.

Step 3 — Inner loop: for each point in region i, compute dot products against all segment vectors, divide by dists to normalise, clip to zero, raise to power p, and accumulate into the affinity row. This triple loop (regions × points × dimensions) is the O(n×m×d) bottleneck and the primary CUDA parallelisation target in Phase 2.

Step 4 — Symmetrize: since the bridge from i to j and from j to i are the same segment, combine both sides: affinity = pow((A + A^T) / counts, 1/p).

Step 5 — Exponential transform: subtract 0.5*max for numerical stability, then apply affinity = exp(γ * affinity) where γ = log(M) / (q90 - q10). This spreads the affinity values so high-affinity pairs dominate clearly over low-affinity pairs. M=1e4 by default means the 90th percentile is 10,000 times larger than the 10th percentile after the transform.

Critical difference from Python

The Python code uses the square root of the raw affinity inside the exponential (redefining affinity as std-dev/length rather than variance/length-squared). Our C++ matches this by taking pow(sym, 1/p) during symmetrisation before the exponential transform, which is mathematically equivalent.

2.3 Spectral Clustering (spectral.hpp / spectral.cpp)

Normalised Laplacian

Given the m×m affinity matrix A, the normalised Laplacian is:

```
L = I - D^(-1/2) * A * D^(-1/2)

where D is the diagonal degree matrix: D[i,i] = sum_j A[i,j]
```

This is computed using Eigen's MatrixXf. Degree values below 1e-10 are treated as zero to avoid numerical issues with isolated nodes.

Eigen decomposition

Eigen's SelfAdjointEigenSolver is used — it exploits the fact that the Laplacian is symmetric to give a more stable and faster decomposition than the general solver. Eigenvalues are returned in ascending order. The first k eigenvectors (corresponding to the k smallest eigenvalues) form the embedding matrix U of shape [m × k].

Row normalisation

Each row of U is L2-normalised before running k-means on it. This is the same step as in the Python code (eigvecs /= np.linalg.norm(eigvecs, axis=1)[:, None]) and improves clustering stability by ensuring all rows have unit length regardless of their absolute magnitude.

Normalised eigengap (ngap)

The eigengap measures how well-defined the clustering is:

```
ngap = (λ[k] - λ[k-1]) / λ[k]
```

A large ngap (close to 1) means there is a clear spectral gap at position k — the k-cluster structure is unambiguous. A small ngap suggests k may not be the right number of clusters, or the data has no clear cluster structure. In the test results above, blobs and moons had ngap ≈ 0.999, confirming near-perfect cluster separation.

Label propagation

After spectral clustering assigns a label to each of the m Voronoi regions, every data point inherits the label of whichever region it was assigned to in the original k-means step. This is the O(n) final step.

2.4 Full pipeline (spectralBridges in spectral.cpp)

The top-level function ties all components together:

```
SBResult spectralBridges(X, n, d, k, m, p, M, n_iter, random_state) {
    1. KMeans(m).fit(X)              -- vector quantization
    2. computeAffinity(X, km, ...)   -- bridge affinity matrix
    3. spectralClustering(aff, m, k) -- Laplacian + eigen + k-means
    4. propagate labels to points    -- O(n) final step
    return labels, eigvals, ngap
}
```

2.5 Correctness validation

Strategy

Since label assignments are arbitrary (cluster 0 in C++ might be cluster 1 in Python), raw label comparison is meaningless. Instead, clustering quality is measured using two metrics that are invariant to label permutation:

- Adjusted Rand Index (ARI) — measures the fraction of point pairs that are correctly grouped together or apart, adjusted for chance. Range: [-0.5, 1], where 1 = perfect agreement.
- Normalised Mutual Information (NMI) — measures information shared between two clusterings, normalised by their average entropy. Range: [0, 1], where 1 = perfect agreement.

Test datasets

Dataset	n	d	k	m	Description
blobs	300	2	3	30	3 isotropic Gaussian clusters, easy case
moons	200	2	2	20	Two interleaving half-circles, non-convex
circles	200	2	2	20	Two concentric circles, non-convex
large	1000	2	5	50	5 overlapping blobs, stress test

Reference generation

tests/generate_reference.py runs the Python SpectralBridges implementation on each dataset with random_state=42 and saves:

- tests/data/{name}_X.csv — the dataset
- tests/data/{name}_labels.csv — Python cluster labels
- tests/data/{name}_true.csv — ground truth labels
- tests/data/{name}_metrics.txt — Python ARI and NMI scores

C++ test (test_reference.cpp)

Loads each CSV, runs the C++ pipeline with the same parameters, computes ARI and NMI both against ground truth and against the Python labels, and checks against minimum thresholds. ARI/NMI implementations are included directly in the test file with no external dependencies.

Actual output

```
=== blobs (n=300, d=2, k=3, m=30) ===
  vs ground truth  -- ARI: 1  NMI: 1
  vs Python output -- ARI: 1  NMI: 1
  Python reference -- ARI: 1  NMI: 1
  Ngap: 0.998797
  Quality check (ARI>=0.85, NMI>=0.85): PASSED

=== moons (n=200, d=2, k=2, m=20) ===
  vs ground truth  -- ARI: 1  NMI: 1
  vs Python output -- ARI: 1  NMI: 1
  Python reference -- ARI: 1  NMI: 1
  Ngap: 0.998744
  Quality check (ARI>=0.7, NMI>=0.7): PASSED

=== circles (n=200, d=2, k=2, m=20) ===
  vs ground truth  -- ARI: 1  NMI: 1
  vs Python output -- ARI: 1  NMI: 1
  Python reference -- ARI: 1  NMI: 1
  Ngap: 0.943079
  Quality check (ARI>=0.7, NMI>=0.7): PASSED

=== large (n=1000, d=2, k=5, m=50) ===
  vs ground truth  -- ARI: 0.973157  NMI: 0.97129
  vs Python output -- ARI: 0.99253   NMI: 0.989127
  Python reference -- ARI: 0.975533  NMI: 0.97318
  Ngap: 0.775606
  Quality check (ARI>=0.8, NMI>=0.8): PASSED

Done.
```

The minor gap on the large dataset (0.973 vs 1.0) is expected: k-means++ is randomised, and the C++ and Python implementations use different random number generators so they explore different initialisations. Both achieve equivalent clustering quality against ground truth.

3 Connection to Phase 2 — Parallelisation

This section briefly describes how the sequential code connects to the upcoming CUDA implementation. The sequential baseline serves two roles in Phase 2: as the timing reference and as the correctness oracle.

3.1 Where the time goes

In the sequential implementation the computational cost breaks down roughly as follows for a large dataset:

Component	Complexity	Parallelisable?
k-means++ seeding	$O(m \times n \times d)$	Partially — seeding is sequential, Lloyd is parallel
Affinity matrix	$O(n \times m \times d)$	Fully — each pair (i,j) is independent
Eigen decomposition	$O(m^3)$	Limited — stays on CPU (m is small)
Label propagation	$O(n)$	Trivially parallel

The affinity matrix computation is the primary GPU target. Each of the m^2 region pairs can be computed independently — no pair depends on another. Within each pair, each point's projection is also independent. This makes it a perfect fit for a CUDA kernel where each thread handles one point-pair dot product.

3.2 What stays the same in Phase 2

The flat row-major Matrix layout (std::vector<float>) chosen for the sequential implementation maps directly to CUDA device memory. No data restructuring is needed — the same arrays can be passed to cudaMemcpy and used inside kernels. The ARI/NMI test infrastructure in test_reference.cpp remains unchanged; Phase 2 results will be validated against the same Python reference files.

3.3 What changes in Phase 2

- affinity.cpp — the triple loop is replaced by a cuBLAS SGEMM call plus a custom CUDA kernel for the clip-and-power step.
- kmeans.cpp — the Lloyd assignment loop ($O(n \times m \times d)$) is offloaded to GPU; the k-means++ seeding remains on CPU.
- CMakeLists.txt — CUDA toolkit and cuBLAS are added as dependencies.

- A new benchmark target measures wall time at varying n and m for both sequential and CUDA versions.

Benchmark goal
Report speedup = (C++ sequential time) / (CUDA time) for the affinity
computation step specifically, and for the full pipeline end-to-end.
Target platforms: NVIDIA RTX 3090 and RTX A5000 (CDS cluster).